

PRACTICAL REFERENCE

Codex Setup Guide

TypeSafe AI skills, Caveman modes, project instructions, and honest token-saving workflows

- 01 Install TypeSafe as a reusable skill
- 02 Use Caveman selectively for concise output
- 03 Keep AGENTS.md short, stable, and project-specific
- 04 Measure total cost, not response length alone

Reusable reference / Windows, macOS, and Linux command line

Edition 1.0 | Verified 19 September 2026

Contents

How to use this guide	3
Scope and assumptions	3
1. The setup at a glance	4
Recommended architecture	4
What not to do	4
2. Install and use the TypeSafe AI skill	4
Project-local installation	5
Global alternative	5
Verify the installation	5
How to invoke it	5
Design principles to preserve	5
3. TypeSafe patterns and prompt examples	5
Architecture prompt	6
Implementation prompt	6
Review prompt	6
4. Install Caveman for Codex	7
Recommended Codex skill install	7
Activate, switch, and stop	7
Optional input-side proxy	7
Privacy and operations	7
5. Choose the right Caveman mode	7
A practical switching rhythm	8
Mode-switch examples	8
6. Recommended project folder structure	8
Why this separation saves context	9
7. A token-conscious AGENTS.md	9
Recommended template	9
High-value token-saving rules	10
8. Practical session workflows	10
Workflow A: new TypeSafe project	10
Workflow B: existing codebase	11
Workflow C: fast maintenance	11
Workflow D: cost diagnosis	11

9. Prompt library11

Start a project 11

Implement with low narration 11

Request deeper reasoning 11

Review token efficiency 12

Stop runaway scope 12

10. Caveats: what token savings really mean12

Measured evidence, with limits 12

Simple evaluation sheet 12

11. Troubleshooting and maintenance13

Update checklist 13

12. One-page quick reference13

Install 13

Modes 13

Three boundaries 14

The default working prompt 14

Sources and further reading14

Document status 14

How to use this guide

Use the quick-start path once, then return to the mode table, AGENTS.md template, and prompt library during real projects. Commands were checked against the upstream TypeSafe skill repository and Caveman documentation on the verification date above.

One-line model

TypeSafe adds structured, typed AI judgments to your application. Caveman changes how concisely the coding agent communicates. AGENTS.md carries stable project rules. They solve different problems and can be used together.

Scope and assumptions

- You are using Codex or another skills-compatible coding agent.
- Node.js and npx are available for skill installation.
- You will inspect third-party skill contents before granting access to sensitive repositories or credentials.
- Examples favor project-local installation for TypeSafe and global installation for a personal response-style skill such as Caveman.

Security note

A skill is instruction text that can influence agent behavior. Review its repository, pin or record versions for controlled environments, and update deliberately. Never assume a popular skill is harmless.

1. The setup at a glance

A clean Codex configuration separates reusable expertise, project policy, and session-level communication style. This keeps instructions understandable and avoids paying repeatedly for large blocks of duplicated guidance.

Layer	Purpose	Typical scope	Example
AGENTS.md	Standing project rules: architecture, commands, quality gates, boundaries.	Repository or subtree	Run unit tests before handoff.
Skill	Reusable specialist workflow or domain knowledge loaded when relevant.	Project-local or global	Design typed TypeSafe judgments.
Session prompt	The task, constraints, desired outcome, and current mode.	Current conversation	Implement ticket routing; use Caveman full.
Tool or proxy	Optional execution or input-side optimization outside the skill text.	Machine/session	Compress repetitive logs before model input.

Recommended architecture

Need	Use	Why
Stable repository conventions	AGENTS.md	Always available; short and authoritative.
Specialized TypeSafe design guidance	TypeSafe skill	Reusable and invoked only when relevant.
Shorter explanations	Caveman lite/full/ultra	Easy to change per session or phase.
Deep explanation or decision record	Normal mode	Preserves nuance when it matters.
Less repetitive tool input	Filtering or an optional proxy	Targets the often-larger input side.

Default recommendation

Install TypeSafe project-locally, install Caveman globally if you want it across repositories, keep Caveman off or lite for discovery, use full for routine implementation, and reserve ultra for tightly scoped work you already understand.

What not to do

- Do not paste an entire third-party SKILL.md into AGENTS.md.
- Do not force ultra-concise responses during architecture review, security work, migrations, or ambiguous debugging.
- Do not equate fewer visible words with lower total cost; tool output, context replay, retries, and rule overhead also consume tokens.

2. Install and use the TypeSafe AI skill

TypeSafe is for application features that need small, typed AI judgments and probabilities: routing, ranking, extraction, verification, state interpretation, and confidence-aware escalation. Code owns the workflow; the model supplies semantic judgment where exact rules are insufficient.

Project-local installation

```
cd path/to/your-project  
  
npx skills add typesafe-ai/skills --skill typesafe-ai
```

Choose Codex when prompted. The upstream installer is project-local by default. A typical result is `.agents/skills/typesafe-ai/SKILL.md`. Project-local installation is easiest to review, version, and share with collaborators.

Global alternative

```
npx skills add typesafe-ai/skills --skill typesafe-ai -g
```

Use a global installation when you want the same TypeSafe guidance across many unrelated projects and accept that updates are machine-specific.

Verify the installation

```
# macOS or Linux  
ls .agents/skills/typesafe-ai  
  
# Windows PowerShell  
Get-ChildItem .agents/skills/typesafe-ai
```

Expected result

The folder should contain `SKILL.md`. Restart or reload Codex if a newly installed skill is not discovered in the current session.

How to invoke it

Natural language is sufficient. Explicitly name the skill when the project depends on it or when you want to ensure the workflow is considered.

```
Use the installed TypeSafe AI skill.  
Design typed judgments for support-ticket routing.  
Return probabilities and route uncertain cases to human review.  
Keep policy, thresholds, and execution in ordinary code.
```

Design principles to preserve

- Ask narrow, coherent questions and give each judgment the state needed to answer.
- Use typed outputs as an interface, not as a guarantee of truth.
- Keep deterministic policy and side effects in code.
- Evaluate thresholds on representative data and consequences.
- Escalate uncertain or high-impact outcomes to a human or deeper reasoning path.
- Read current TypeSafe API or SDK docs before writing version-dependent integration code.

3. TypeSafe patterns and prompt examples

TypeSafe works best when an open-ended language task is decomposed into bounded judgments that software can combine. The following map is intentionally conceptual; consult live documentation for current API names and SDK details.

Pattern	Judgment	Code remains responsible for
Routing	Choose the best handler or department.	Allowed routes, access control, dispatch, audit.
Ranking	Score or compare candidates against a defined dimension.	Candidate retrieval, weights, tie rules, presentation.
Extraction	Select a source value or span from known candidates.	Validation, normalization, persistence.
Verification	Judge whether a field or claim is supported by evidence.	Evidence collection, thresholds, escalation.
Interactive flow	Choose the next bounded action from current state.	State storage, freshness checks, side effects.

Architecture prompt

Create a FastAPI service for inbound support tickets.
Use the installed TypeSafe skill where semantic judgment helps.

Requirements:

- route to billing, technical, account, or other
- return typed probabilities
- require human review below an evaluated threshold
- keep authentication, policy, and side effects in code
- begin with architecture and test cases; then implement incrementally

Implementation prompt

Implement the approved ticket-routing design.
Use TypeSafe for the department judgment only.
Batch independent questions over the same state when useful.
Add representative tests for ambiguous and no-match cases.
Do not invent current SDK methods: check the live docs first.
Run tests and report only decisions, failures, and changed files.

Review prompt

Review this TypeSafe integration. Check:

1. state contains the evidence needed for each judgment
2. each question is narrow and typed
3. probabilities are used with explicit policy
4. uncertain cases have a safe path
5. credentials stay server-side
6. tests cover model, code, and service failures

Return findings by severity with file and line references.

Avoid the generic classifier trap

Before asking for a label, consider whether the feature really needs a choice, independent yes/no judgments, a graded score, candidate selection, evidence verification, or a staged combination.

4. Install Caveman for Codex

Caveman is a third-party concise-response skill. Its small form primarily reduces prose output. The project also offers an optional local proxy intended to compress input such as logs, JSON, diffs, and search results. Treat these as separate products with different risk and benefit profiles.

Recommended Codex skill install

```
npx skills add JuliusBrussee/caveman --skill '*' -a codex --yes -g
```

The shorter upstream quick-start form is also available:

```
npx skills add JuliusBrussee/caveman -g
```

Use a global install for a personal communication style you want across projects. Omit `-g` if you prefer a repository-local copy. If installation behavior changes, consult the upstream README and INSTALL.md.

Activate, switch, and stop

Command or phrase	Effect	Best use
<code>/caveman lite</code>	Concise but grammatical.	Planning, collaboration, unfamiliar code.
<code>/caveman full</code>	Aggressively concise; current default style.	Routine build/test/fix loops.
<code>/caveman ultra</code>	Highly compressed wording.	Known, narrow implementation tasks.
<code>/caveman off</code>	Turn Caveman off.	Deep explanation, high-stakes review.
<code>normal mode</code>	Return to ordinary prose.	Natural-language fallback across agents.

Optional input-side proxy

```
npm install -g @caveman-ai/cli
caveman setup --install
caveman codex
```

Install separately

The response-style skill and the proxy can be used independently. Start with the skill. Consider the proxy only after measuring that large, repetitive inputs are a real cost driver for your workload.

Privacy and operations

The Caveman project states that compression runs locally and originals are recoverable, but its CLI enables anonymous usage statistics by default. Review SECURITY.md and your organization's policy before deployment. The project documents `caveman telemetry off` and `DO_NOT_TRACK=1` as opt-out mechanisms.

5. Choose the right Caveman mode

Select a mode by the cost of misunderstanding, not merely by your desire for short answers. Change modes as the work changes.

Situation	Suggested mode	Reason
Repository orientation	Normal or lite	You need names, relationships, assumptions, and caveats.
Architecture decision	Normal	Tradeoffs and rejected options are part of the deliverable.
Routine implementation	Full	Actions and verification matter more than narration.
Known mechanical edit	Ultra	The task is bounded and easy to verify.
Debugging an unclear failure	Lite or normal	Premature compression can hide the chain of evidence.
Security, data loss, migration	Normal	Risk, rollback, and uncertainty must remain explicit.
Test/fix repetition	Full or ultra	Keep only failures, fixes, and final status.
Handoff or decision record	Normal	Future readers need durable context.

A practical switching rhythm

1. Begin in normal or lite while defining scope and architecture.
2. Move to full after decisions are settled and implementation becomes routine.
3. Use ultra only for narrow, reversible tasks with strong tests or easy inspection.
4. Return to normal for explanations, approvals, risky operations, or final handoff.

Safety invariant

Conciseness must never suppress security warnings, irreversible-action confirmations, exact error messages, commands, file paths, or material uncertainty.

Mode-switch examples

```
normal mode
Explain the architecture, alternatives, and risks.

/caveman full
Implement the approved plan. Run tests. Fix failures.

/caveman ultra
Rename the validated symbols only. Run the focused test.

normal mode
Summarize the final behavior, residual risks, and rollback.
```

6. Recommended project folder structure

Keep skills distinct from project instructions and source code. Exact discovery paths can vary by installation method and Codex version, so verify the location created by your installer.

```
your-project/
|-- AGENTS.md
|-- .agents/
|  |-- skills/
|  |-- typesafe-ai/
|  |-- SKILL.md
|-- docs/
|  |-- architecture.md
|  |-- decisions/
|-- src/
|-- tests/
|-- .env.example
`-- README.md
```

Location	Put here	Do not put here
AGENTS.md	Stable repo commands, boundaries, conventions, quality gates.	Copied skill bodies, secrets, transient task notes.
.agents/skills/	Reusable skill packages with their own SKILL.md.	Project source code or generated credentials.
docs/	Architecture, decision records, operating procedures.	Rules that must always govern the agent unless AGENTS.md links them clearly.
.env / secret store	Credentials outside version control.	API keys in AGENTS.md, prompts, examples, or test fixtures.

Why this separation saves context

AGENTS.md is part of the standing project context. A long file is repeatedly expensive and harder to maintain. A focused skill can be selected when the task matches it. Task-specific detail belongs in the current request or a linked artifact, not in permanent global instructions.

Rule of thumb

If an instruction is true for nearly every task in this repository, it may belong in AGENTS.md. If it teaches a reusable specialty, it belongs in a skill. If it is true only for today's task, put it in the prompt or ticket.

7. A token-conscious AGENTS.md

The best AGENTS.md is short, concrete, and testable. It should reduce rediscovery and rework without becoming a second handbook.

Recommended template

```
# Project instructions

## Scope
- Work only in this repository unless explicitly asked.
- Preserve unrelated user changes.

## Architecture
- Keep domain logic independent of HTTP and persistence.
- Put external integrations behind interfaces.

## Commands
- Install: <one canonical command>
- Test: <fast test command>
- Full check: <lint + typecheck + tests>

## Working style
- Inspect before editing.
- Prefer the smallest complete change.
- Do not narrate routine file reads or successful commands.
- Report decisions, failures, risks, and verification results.
- Ask only when a missing choice materially changes the result.

## TypeSafe
- For typed semantic decisions, routing, ranking, extraction,
  verification, or confidence-aware workflows, use the installed
  typesafe-ai skill and current TypeSafe documentation.
- Keep deterministic policy and side effects in code.

## Quality gates
- Add or update focused tests for behavior changes.
- Never claim success without running the relevant checks.
```

High-value token-saving rules

Rule	What it prevents
Do not narrate routine reads or successful commands.	Low-value progress prose.
Report only decisions, failures, risks, and verification.	Verbose summaries that repeat tool output.
Use the smallest relevant test first.	Large noisy logs and slow feedback loops.
Search narrowly before opening whole files.	Unnecessary input context.
Stop after the requested outcome is verified.	Unrequested expansion and additional tool calls.
Link detailed procedures instead of duplicating them.	Repeated permanent instructions.

Do not over-compress policy

Keep exceptions, safety boundaries, exact commands, and acceptance criteria explicit. Removing the words that disambiguate a rule can cost more through retries than it saves in context.

8. Practical session workflows

Workflow A: new TypeSafe project

1. Install the TypeSafe skill project-locally and verify SKILL.md.
2. Start in normal mode. Describe user behavior, data, risks, and constraints.
3. Ask Codex to identify deterministic rules versus semantic judgments.

4. Approve the architecture, confidence policy, and tests.
5. Switch to Caveman full for implementation and verification.
6. Return to normal for the handoff and unresolved-risk summary.

Workflow B: existing codebase

```
/caveman lite
Inspect the current routing flow. Identify the smallest boundary where a
typed semantic judgment replaces prompt-and-parse behavior. Do not edit yet.

normal mode
Explain the proposed interface, uncertainty policy, migration risk, and tests.

/caveman full
Implement the approved boundary. Run focused tests, then the full check.
Stop when verified.
```

Workflow C: fast maintenance

```
/caveman ultra
Fix the failing null-handling test only. Preserve public behavior.
Run the focused test. If the failure has a different root cause, stop and
switch to normal mode with evidence.
```

Workflow D: cost diagnosis

1. Measure several representative sessions, not one unusually verbose answer.
2. Separate input, output, cached, tool, and retry costs when the provider exposes them.
3. Compare normal, lite, and full on the same task and quality bar.
4. If inputs dominate, reduce noisy reads or evaluate an input-side compressor.
5. Keep the configuration only if it reduces total cost without increasing failure or review time.

9. Prompt library

Start a project

```
Create this project using the repository's AGENTS.md.
Use the installed TypeSafe skill where a typed semantic judgment is useful.
Use ordinary code for exact rules, validation, policy, and side effects.
Start by proposing architecture, data flow, confidence handling, and tests.
Do not implement until the design is internally consistent.
```

Implement with low narration

```
/caveman full
Implement the approved plan. Preserve unrelated changes.
Use focused searches and tests first.
Do not explain routine reads or successful commands.
Report changed behavior, verification, and remaining risks.
```

Request deeper reasoning

```
normal mode
Explain why this judgment should use TypeSafe instead of deterministic code
or a conventional generative model. Compare accuracy, latency, cost,
uncertainty handling, observability, and failure modes.
```

Review token efficiency

```
Review this repository's AGENTS.md and session workflow for token waste.
Preserve every safety boundary and acceptance criterion.
Identify duplicated, transient, or non-testable instructions.
Propose a shorter version and estimate which repeated context it removes.
```

Stop runaway scope

```
Work only on the stated acceptance criteria.
Do not refactor adjacent code unless required for correctness.
If a broader change becomes necessary, stop and explain the evidence,
impact, and smallest viable option.
```

10. Caveats: what token savings really mean

Visible response length is only one component of usage. Agentic coding sessions may spend far more tokens rereading repository context, tool schemas, logs, diffs, and test output. A concise style can still be valuable, but it should be evaluated in the full workflow.

Claim	Practical interpretation
Shorter responses save tokens.	Usually true for output, but the skill's instructions add input overhead.
Ultra is always cheapest.	Not if missing context causes clarification, mistakes, or reruns.
A skill reduces model reasoning.	A response-style skill mainly changes expression; it does not guarantee lower internal or tool-side usage.
Compression is lossless in practice.	Original bytes may be recoverable, but the model can still miss context that was not surfaced initially.
One benchmark predicts my bill.	Workload mix, provider pricing, caching, model, and agent behavior all matter.
Typed output is correct output.	A schema guarantees shape, not truth; validate domain performance and escalation policy.

Measured evidence, with limits

The Caveman repository currently summarizes third-party and project-run evaluations. It reports that a JetBrains test of 86 coding tasks found 8.5% fewer output tokens with no detected quality change, while an Adobe paper found larger reductions in selected chat-style conditions. The same repository warns that skill rules consume input tokens, that agentic coding costs are often input-heavy, and that compressing the user's prompt can worsen results. Treat these as evidence to investigate, not a guarantee for your workload.

Best metric

Compare end-to-end task cost at the same acceptance criteria: successful completion, review effort, latency, retries, and total billed tokens. A shorter answer that creates one extra debugging cycle is not a saving.

Simple evaluation sheet

Variant	Success?	Input	Output	Retries	Human minutes
Normal					
Lite					
Full					
Ultra					

11. Troubleshooting and maintenance

Symptom	Check	Action
Skill is not discovered	Install path, SKILL.md presence, current session.	Reload Codex; reinstall for the intended agent and scope.
TypeSafe guidance is not applied	Task language never mentions typed decisions or the skill.	Say 'Use the installed TypeSafe AI skill' in the prompt.
Caveman does not activate	Global vs local install; command support in current Codex surface.	Try '/caveman full' or 'Use the Caveman skill in full mode.'
Responses are too terse	Mode is full or ultra during discovery.	Switch to lite or normal mode.
Token usage remains high	Input/tool output dominates; repeated wide file reads.	Narrow searches/tests; measure input-side causes before adding tools.
Agent repeats project rules	AGENTS.md is verbose or duplicated in prompts.	Keep one authoritative copy and ask for action, not recitation.
Update changes behavior	Moving upstream version or installer defaults.	Review diffs, record versions, and test on a non-critical task.

Update checklist

- Read upstream release notes or repository changes.
- Review the installed SKILL.md diff before accepting it.
- Confirm installation scope and discovery path.
- Run a representative task and compare quality plus total usage.
- Update this guide or internal runbook when commands or modes change.

Version note

This guide is current as of 19 September 2026. Third-party installation commands, defaults, licensing, telemetry, and supported surfaces may change. The linked upstream sources remain authoritative.

12. One-page quick reference

Install

```
# TypeSafe, project-local
npx skills add typesafe-ai/skills --skill typesafe-ai

# Caveman, global for Codex
npx skills add JuliusBrussee/caveman --skill '*' -a codex --yes -g
```

Modes

Mode	Use it for	Avoid it for
Normal	Architecture, explanation, risk, handoff	Routine narration
Lite	Discovery, debugging, collaboration	Maximum compression
Full	Build, test, fix, review loops	Ambiguous high-stakes work
Ultra	Known mechanical edits	New systems, migrations, security

Three boundaries

- AGENTS.md: short, stable, project-specific rules.
- Skills: reusable specialist workflows; do not paste them into AGENTS.md.
- Prompt: today's outcome, constraints, scope, and chosen mode.

The default working prompt

```
/caveman full
Use AGENTS.md and the installed TypeSafe skill where appropriate.
Implement the requested behavior with the smallest complete change.
Keep exact rules and side effects in code.
Run focused tests, then the required full check.
Report behavior, verification, and remaining risks only.
```

Remember

Start clear, compress during execution, expand again for risk and handoff. Measure end-to-end outcomes.

Sources and further reading

Primary project sources consulted for this edition. Accessed 19 September 2026.

1. TypeSafe Agent Skills repository. Installation, scope, and usage. <https://github.com/typesafe-ai/skills>
2. TypeSafe live documentation index. Current concepts, primitives, SDKs, confidence, and cookbooks. <https://docs.typesafe.ai/llms.txt>
3. Caveman repository README. Installation, modes, proxy, benchmarks, privacy, and caveats. <https://github.com/JuliusBrussee/caveman>
4. Caveman installation guide. Agent-specific installation and activation. <https://github.com/JuliusBrussee/caveman/blob/main/INSTALL.md>
5. Caveman honest numbers. Measurement scope, overhead, and limitations. <https://github.com/JuliusBrussee/caveman/blob/main/docs/HONEST-NUMBERS.md>

Document status

This is an operational reference, not a vendor guarantee. Commands are presented for convenience; always review current upstream instructions and your organization's security requirements before installing third-party code or routing sensitive data through new tooling.